

Spec Driven Development · Projeto 1

Controle de Gastos Pessoais

Relatório da sessão — esqueleto do projeto e implementação da Aula 1
(registrar e listar transações)

Python 3.11+

FastAPI

SQLite

Pydantic

CONTEXTO

De onde partimos

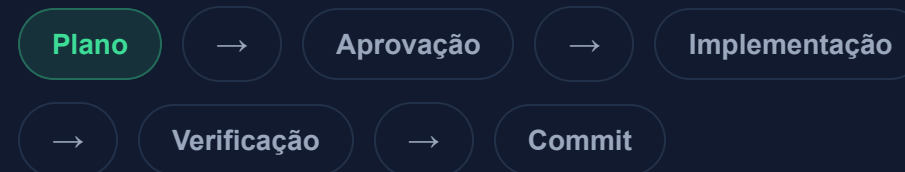
JÁ EXISTIA

- `specs/` com as specs das 6 aulas do projeto
- `docs/prompts-projeto1-controle-gastos.md`, o roteiro de prompts da aula
- `.claude/commands/implementar-spec.md`, o slash command `/implementar-spec`
- Nenhum código da aplicação ainda

OBJETIVO DA SESSÃO

- Criar o esqueleto do projeto + `CLAUDE.md`
- Implementar a Aula 1: `specs/aula-1-transacoes.md`
- Verificar critérios de aceite e commitar

Fluxo seguido em cada etapa:



PASSO 1

Esqueleto do projeto + CLAUDE.md

ESTRUTURA CRIADA

```
SDD1/ └─ CLAUDE.md └─ requirements.txt └─  
.gitignore └─ app/ | └─ main.py → app + startup +  
handler 422 | └─ database.py → conexão + init_db()  
| └─ models.py → schemas Pydantic | └─ routers/  
└─ docs/ (já existia) └─ specs/ (já existia) └─  
.claude/ (já existia)
```

CLAUDE.MD DOCUMENTA

- Descrição do projeto e stack
- Como rodar: venv + `pip install` + `uvicorn --reload`
- Estrutura de pastas e papel de cada módulo
- Convenções (datas, valores, tipo, erro 422)
- Seção "Decisões registradas" para acumular ao longo das aulas

Convenções e arquitetura

CONVENÇÕES DO PROJETO

- Datas em ISO `YYYY-MM-DD`
- Valores monetários com 2 casas decimais
- `tipo` sempre `"entrada"` ou `"saida"`
- Erros de validação em HTTP 422, corpo `{ "erro": "..."}`
- Banco SQLite criado na inicialização, se não existir

DECISÕES TÉCNICAS

- Acesso via `sqlite3` puro (sem ORM) — controle direto do SQL para filtros dinâmicos e agregações das próximas aulas
- Erro 422 tratado globalmente com um `exception_handler` de `RequestValidationError` — não repetido rota a rota
- `init_db()` chamado no evento `startup` do FastAPI

Registrar e listar transações

ENDPOINTS IMPLEMENTADOS

- `POST /transacoes` → 201 com a transação criada (com `id`)
- `GET /transacoes` → lista ordenada por `data` decrescente

MODELO DE DADOS

- `id` (auto), `data`, `descricao`, `valor`, `tipo`

ARQUIVOS ALTERADOS

- `app/database.py` — tabela `transacoes`, `criar_transacao`, `listar_transacoes`
- `app/models.py` — `TransacaoCreate`, `Transacao`
- `app/routers/transacoes.py` — rotas POST/GET (novo)
- `app/main.py` — registra o router

PASSO 3

Verificação contra os critérios de aceite

CRITÉRIO	RESULTADO
POST /transacoes retorna 201 com o objeto e id	✓ Passou
GET /transacoes lista mais recentes primeiro	✓ Passou
Transações sobrevivem ao reinício do servidor	✓ Passou
Erros de validação em 422 no formato {"erro": "..."}	✓ Passou

Testado ponta a ponta: servidor subido, transações criadas com datas diferentes, servidor reiniciado e histórico conferido, além de 3 cenários inválidos (valor ≤ 0 , tipo inválido, data malformada).

Versionando o slash command

PROBLEMA ENCONTRADO

O repositório tem um `.gitignore` na raiz que ignora `.claude/` em **todo** o repositório — política já existente, usada para não versionar configs locais.

Isso também excluía `SDD1/.claude/commands/implementar-spec.md`, que não é config local: é parte do fluxo de trabalho do projeto.

SOLUÇÃO

Exceção adicionada ao `.gitignore` raiz, escopada só a este projeto:

→ Reabre `SDD1/.claude/`

→ Reignora tudo dentro, exceto `commands/`

Resto do repositório continua ignorando `.claude/` normalmente.

HISTÓRICO

Commits desta sessão

648b7c7

Versiona o slash command /implementar-spec do projeto SDD1

2 arquivos alterados, 13 inserções

d9212f5

Adiciona esqueleto do projeto Controle de Gastos e Aula 1 (transacoes)

16 arquivos alterados, 649 inserções

Projeto pronto para uso

RODANDO LOCALMENTE

- Servidor em `http://127.0.0.1:8000`, com `--reload`
- Documentação interativa em `/docs`
- `gastos.db` criado automaticamente, ignorado no Git
- `.venv/` com as dependências instaladas

COMO SUBIR DE NOVO

```
source .venv/bin/activate uvicorn app.main:app --  
reload
```

Testar:

```
curl -X POST /transacoes -d '{...}' curl /transacoes
```

PRÓXIMOS PASSOS

Aula 2 — CRUD completo com validação

- 1 **Rodar** `/implementar-spec specs/aula-2-crud-validacao.md` (edição, exclusão e validação de escrita).
- 2 **Revisar o plano** antes de implementar — ponto mais barato para corrigir rumo.
- 3 **Verificar** os critérios de aceite item a item e registrar decisões novas no `CLAUDE.md`.
- 4 **Commit por aula** — cada incremento fechado vira um commit, spec incluída.